



Protocolo de implementación de NetSegment API

Grupo 13 Versión 1.0.0

Código	Apellidos y Nombres	Fecha
2024001848	Acosta Huamán, Daniel Armando	04/10/2026
2024000333	Benites Castro, Arturo José	04/10/2026
2024000178	Limache Quispe, Felix Fabricio	04/10/2026
2024001212	Mollo Huayhua, Luis Felipe	04/10/2026
2025002762	Rodríguez Fádel, Gian Piero Khalil	04/10/2026
2024000462	Vega Mamani, Anthony Yerson	04/10/2026

Este protocolo documenta la instalación, configuración, ejecución y validación de NetSegment API 1.0.0 del grupo 13. Permite reproducir la implementación a partir del código fuente, las dependencias y los archivos de configuración de la solución. La versión implementa cálculos IPv4 de FLSM, VLSM y agregación CIDR. El procedimiento principal utiliza Linux y Python 3.12; la instalación persistente en Ubuntu Server se presenta después de la ejecución local.

a Información general

Nombre del proyecto: Desarrollo de un motor de microservicio para segmentación, asignación y agregación de direccionamiento IPv4 en entornos de red locales.

Nombre del sistema: NetSegment API. Número de grupo: 13. Institución: Universidad Católica de Santa María. Escuela Profesional: Ingeniería de Sistemas. Docente: Javier Fernando Angulo Osorio.

Curso: Computación en Red III. Secciones del equipo: B y C. Fecha de elaboración: 4 de octubre de 2026. Versión del sistema: 1.0.0.

Repositorio del proyecto: https://github.com/DanielUCSM/X_NetSegment-API

Nombre establecido por la estructura de entrega: 13_NetSegmentAPI. La configuración del nombre y la invitación al docente se describen en la sección f.

b Descripción general de la solución

La planificación manual del direccionamiento puede producir máscaras incorrectas, rangos superpuestos o un aprovechamiento inadecuado de los bloques disponibles. NetSegment API automatiza estos cálculos y expone los resultados mediante una API REST que puede consumir un navegador, curl, un script u otra aplicación.

El objetivo del sistema es producir planes de direccionamiento IPv4 válidos a partir de la red base y de los requerimientos de subredes o hosts. FLSM divide una red en bloques iguales; VLSM asigna bloques de tamaño variable de mayor a menor demanda; CIDR resume únicamente conjuntos que forman una superred exacta.

Los usuarios previstos son estudiantes, docentes, administradores de redes y desarrolladores de herramientas de infraestructura. La solución es un microservicio backend stateless: recibe JSON, realiza el cálculo y responde sin almacenar sesiones ni resultados. No configura routers, no descubre dispositivos y no modifica redes reales.

La versión 1.0.0 implementa IPv4. Aunque el título original del dossier menciona IPv4/IPv6, el contrato proporcionado solo define operaciones IPv4 y el soporte IPv6 no está implementado. La interfaz cliente incluida es Swagger UI; no hay aplicación móvil, escritorio o frontend propio.

c Arquitectura y tecnologías utilizadas

c1 Arquitectura del sistema

El cliente envía una solicitud HTTP al servidor ASGI Uvicorn. FastAPI selecciona la ruta y Pydantic valida el cuerpo. El motor de cálculo utiliza IPv4Network, IPv4Address y collapse_addresses del módulo estándar ipaddress. La separación entre rutas, modelos y servicios permite verificar los algoritmos sin incorporar persistencia.

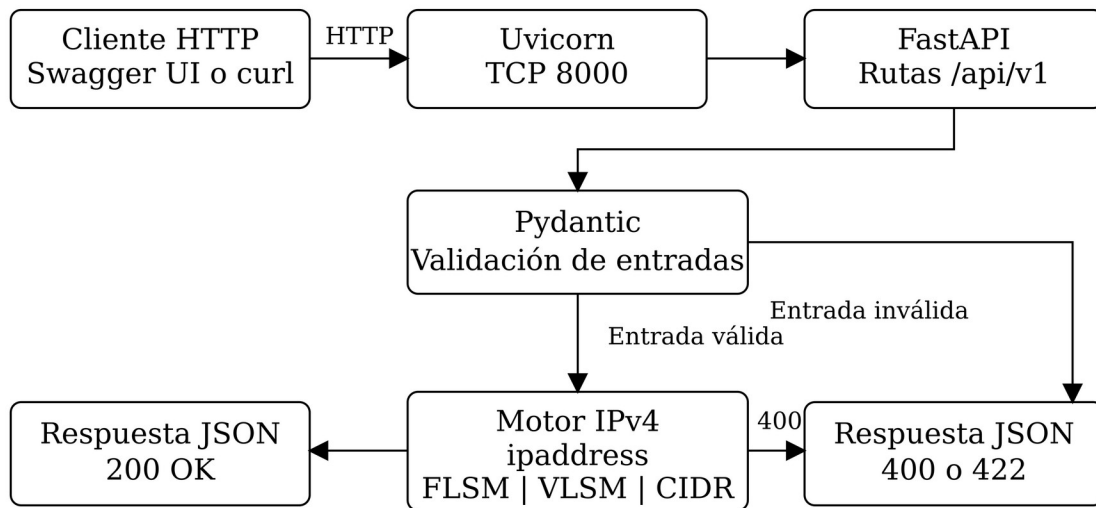


Figura 1. Arquitectura y flujo de validación de NetSegment API. Fuente: elaboración a partir del código implementado.

Una entrada inválida produce HTTP 422. Un requerimiento que supera la capacidad o una agregación no válida produce HTTP 400. La respuesta exitosa contiene status igual a success y un objeto data. GET /health informa disponibilidad y versión.

c2 Tecnologías y versiones

Componente	Versión	Uso
Python	3.12 recomendado	Lenguaje; validado con 3.12.14
FastAPI	0.142.2	Rutas HTTP y documentación OpenAPI
Pydantic	2.13.5	Validación de solicitudes
Uvicorn	0.54.0	Servidor ASGI
Starlette	1.7.0	Infraestructura HTTP de FastAPI

Componente	Versión	Uso
python-dotenv	1.2.4	Lectura del archivo .env
ipaddress	Biblioteca estándar	Operaciones sobre redes IPv4
pytest	9.1.1	Pruebas automatizadas
httpx	0.28.1	Cliente de las pruebas HTTP
Git y GitHub	Sin versión de servidor fijada	Código y colaboración

requirements.txt fija dependencias de ejecución directas y transitivas. requirements-dev.txt incorpora las herramientas de prueba. No se requiere motor de base de datos, servidor SQL, servicio de almacenamiento ni API externa. La conexión a Internet se usa para clonar e instalar dependencias; los cálculos se realizan localmente.

El entorno recomendado de despliegue es Ubuntu Server 24.04 LTS con systemd. La verificación efectuada corresponde a Linux x86_64 y Python 3.12.14; no se declara probado el despliegue remoto ni otros sistemas operativos.

c3 Organización del código

Archivo o directorio	Responsabilidad
app/main.py	Aplicación FastAPI, rutas, disponibilidad y errores
app/schemas.py	Validación de redes, enteros y departamentos
app/services.py	Cálculos FLSM, VLSM y CIDR
app/__main__.py	Carga de configuración e inicio de Uvicorn
app/__init__.py	Versión del sistema
examples/	Cuerpos JSON de referencia
tests/test_api.py	Casos de prueba automatizados
scripts/smoke_test.py	Comprobación contra el servidor real
deploy/netsegment.service	Configuración persistente con systemd
docs/	Protocolo, diagrama, contrato y registro de validación

d Requisitos para la implementación

d1 Hardware y software

Recurso	Mínimo orientativo	Recomendado
CPU	1 núcleo	2 núcleos
RAM disponible	512 MB	2 GB
Disco libre	500 MB	5 GB en SSD

Recurso	Mínimo orientativo	Recomendado
Sistema operativo	Linux con Python 3.12	Ubuntu Server 24.04 LTS
Red	Acceso para descargas	Conexión estable para clientes autorizados

Estos valores proceden del diseño del dossier y orientan la instalación de laboratorio. No son resultados de una prueba de carga. Se requiere una terminal, permisos para instalar herramientas del sistema, Git, Python, pip y venv. El navegador y curl permiten comprobar la API. Uvicorn es el servidor necesario; Nginx no es obligatorio para la ejecución local.

d2 Variables de entorno y puertos

Variable	Valor de ejemplo	Función
HOST	127.0.0.1	Dirección de escucha local
PORT	8000	Puerto TCP; rango permitido 1 a 65535
LOG_LEVEL	info	Nivel de registro de Uvicorn

El archivo .env.example contiene únicamente configuración de ejemplo. Se copia a .env, que está excluido de Git. El comando `python -m app` carga ese archivo desde la raíz del proyecto. Una variable ya definida en el proceso tiene prioridad sobre .env.

TCP 8000 es el puerto predeterminado. HOST igual a 127.0.0.1 permite conexiones únicamente desde el mismo equipo. Para un laboratorio compartido puede usarse una dirección de la interfaz o 0.0.0.0, con control de acceso en la infraestructura y un puerto autorizado. 0.0.0.0 es una dirección de escucha; el cliente debe conectarse a la IP real del servidor.

La aplicación no utiliza contraseñas, tokens, certificados ni credenciales de base de datos. HTTPS y los puertos 80/443 corresponden a una eventual infraestructura de proxy y no al despliegue local validado. La API no incorpora autenticación; una publicación en Internet exige que la infraestructura implemente HTTPS, autenticación y límites de tráfico.

e Procedimiento de implementación

e1 Instalar las herramientas

Ejecutar en Ubuntu 24.04 o una distribución compatible:

```
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip curl unzip
python3 --version
git --version
```

Resultado esperado: herramientas instaladas y Python 3.12 disponible. Si la distribución proporciona otra versión, instalar Python 3.12 con su gestor antes de crear el entorno; no reutilizar un entorno creado con otro intérprete.

e2 Descargar el proyecto e identificar la versión

Para instalar desde el paquete NetSegment_API_Grupo_13.zip, extraerlo en un directorio nuevo:

```
unzip NetSegment_API_Grupo_13.zip
cd 13_NetSegmentAPI
```

Resultado esperado: código, dependencias, ejemplos y documentación disponibles en 13_NetSegmentAPI. Usar ese directorio como raíz en los pasos siguientes. La extracción no genera un historial Git. La versión instalada se identifica por app/__init__.py y /health.

Para obtener una copia con historial Git, utilizar:

```
git clone https://github.com/DanielUCSM/X_NetSegment-API.git netsegment
cd netsegment
git branch --show-current
git rev-parse HEAD
```

Resultado esperado: copia local en netsegment, rama main y un identificador completo de commit. Registrar ese identificador durante la demostración para relacionar la ejecución con el código entregado. Después del cambio de nombre, sustituir la URL por la del repositorio 13_NetSegmentAPI; el directorio local puede seguir llamándose netsegment.

e3 Crear y activar el entorno virtual

```
python3 -m venv .venv
source .venv/bin/activate
python --version
python -m pip --version
```

Resultado esperado: directorio .venv y comandos asociados a ese entorno. La ruta de pip debe contener .venv. Mantener el entorno activado para los pasos siguientes.

e4 Instalar dependencias

```
python -m pip install -r requirements.txt
python -m pip check
```

Resultado esperado: instalación de las versiones fijadas y mensaje No broken requirements found. Si falla una descarga, resolver la conexión y repetir la instalación; no eliminar las restricciones de versión para ocultar el problema.

e5 Base de datos y configuraciones iniciales

No corresponde crear una base de datos. El sistema no contiene modelos persistentes, scripts SQL, migraciones ni datos semilla. Todas las entradas necesarias se envían en cada solicitud.

Resultado esperado: los cálculos funcionan sin servicios SQL y sin directorios de datos persistentes. No se debe instalar un motor de base de datos para completar este paso.

e6 Configurar las variables de entorno

```
cp .env.example .env
```

El contenido local de ejemplo es:

```
HOST=127.0.0.1
PORT=8000
LOG_LEVEL=info
```

Resultado esperado: archivo .env en la raíz, con un puerto libre y sin credenciales. Cuando se trabaja en una copia con historial Git, el archivo no aparece como pendiente de commit porque .gitignore lo excluye. En esa copia, comprobarlo con:

```
git check-ignore .env
```

Resultado esperado en una copia Git: salida .env. Esta comprobación se utiliza en un repositorio clonado; el paquete ZIP no incluye historial Git. Si el puerto está ocupado, editar PORT y utilizar el mismo valor en los comandos y en el navegador.

e7 Compilación del proyecto

Python ejecuta el código sin un proceso de construcción previo. La instalación no necesita npm, un bundle de frontend ni compilación nativa de la aplicación. De forma opcional puede verificarse la sintaxis:

```
python -m compileall -q app
```

Resultado esperado: salida sin errores. Los archivos de caché generados quedan excluidos del repositorio. Esta comprobación no reemplaza las pruebas funcionales.

e8 Ejecutar el backend

```
python -m app
```

Resultado esperado: mensajes de inicio de Uvicorn y escucha en `http://127.0.0.1:8000` con la configuración de ejemplo. Mantener abierta la terminal; Ctrl+C detiene el proceso.

Para editar código durante el desarrollo puede iniciarse una sesión alternativa, después de detener la anterior:

```
python -m uvicorn app.main:app \
--host 127.0.0.1 --port 8000 --reload
```

Resultado esperado: reinicio automático ante cambios. Este comando usa argumentos y no carga el .env del proyecto. La recarga automática es solo para desarrollo.

e9 Ejecutar la interfaz cliente

Abrir `http://127.0.0.1:8000/docs` en un navegador. La página se sirve desde FastAPI y no requiere ejecutar un frontend separado. Seleccionar una ruta POST, pulsar Try it out, escribir el JSON del ejemplo y pulsar Execute. /redoc ofrece otra vista de documentación y /openapi.json entrega el contrato.

Resultado esperado: rutas visibles y respuestas de la API en la misma página. curl o scripts propios pueden utilizarse como clientes alternativos. Una aplicación cliente futura requerirá su propio procedimiento; no forma parte de esta versión.

e10 Comprobar disponibilidad y operaciones

Abrir una segunda terminal en la raíz de la copia instalada y activar .venv. Los siguientes comandos se ejecutan desde la raíz del proyecto mientras el backend continúa iniciado:

```
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
python scripts/smoke_test.py
```

Resultado esperado: status ok, versión 1.0.0 y seis comprobaciones HTTP correctas. Si se cambió el puerto, indicar la URL al script, por ejemplo `python scripts/smoke_test.py http://127.0.0.1:8001`.

Prueba FLSM:

```
curl -fsS -X POST \
  http://127.0.0.1:8000/api/v1/subnetting/flsm \
  -H 'Content-Type: application/json' \
  --data @examples/flsm.json
```

Resultado esperado: HTTP 200, cuatro subredes /26 y 62 hosts utilizables por subred. Sus redes son 192.168.10.0, 192.168.10.64, 192.168.10.128 y 192.168.10.192. La última dirección broadcast es 192.168.10.255.

Prueba VLSM:

```
curl -fsS -X POST \
  http://127.0.0.1:8000/api/v1/subnetting/vlsm \
  -H 'Content-Type: application/json' \
  --data @examples/vlsm.json
```

Resultado esperado: HTTP 200 y la siguiente distribución, en orden descendente de demanda:

Departamento	Hosts solicitados	Bloque asignado	Hosts utilizables
Ingenieria	500	172.16.0.0/23	510
Ventas	120	172.16.2.0/25	126
Servidores	50	172.16.2.128/26	62
Enlace-WAN	2	172.16.2.192/30	2

La red /22 tiene 1024 direcciones. Los bloques ocupan 708 y quedan 316 sin asignar. La capacidad utilizable de hosts suma 700, frente a 672 hosts pedidos; quedan 28 espacios de hosts dentro de los bloques asignados. Estas magnitudes corrigen el total 772 y el resto 252 del ejemplo del dossier.

Prueba de agregación CIDR:

```
curl -fsS -X POST \
  http://127.0.0.1:8000/api/v1/supernetting/aggregate \
  -H 'Content-Type: application/json' \
  --data @examples/aggregate.json
```

Resultado esperado: HTTP 200, ruta 192.168.0.0/22, máscara 255.255.252.0, wildcard 0.0.3.255 y reducción de cuatro rutas a una, equivalente al 75 %. La agregación no incluye direcciones ajenas a las redes recibidas.

e11 Validar entradas y errores

El script de comprobación ya incluye una IPv4 inválida y una solicitud que excede la capacidad. Para observar manualmente un error de sintaxis:

```
curl -sS -i -X POST \
  http://127.0.0.1:8000/api/v1/subnetting/flsm \
  -H 'Content-Type: application/json' \
  -d '{"network":"192.168.1.300/24","subnets_needed":4}'
```

Resultado esperado: HTTP 422 y detalle de validación. El comando no usa -f para permitir ver el cuerpo del error. Los errores lógicos usan HTTP 400 y los campos status, error_code y message. Una ruta inexistente produce 404.

Las redes deben ser direcciones base exactas con prefijo CIDR. Los valores enteros son estrictos: no se aceptan cadenas ni booleanos como cantidad. No se permiten campos adicionales. Los nombres de departamentos no pueden repetirse. FLSM puede devolver más subredes que las solicitadas al completar una potencia de dos. FLSM y VLSM asignan como mínimo /30, reservando red y broadcast. CIDR exige continuidad, ausencia de solapamiento y alineamiento exacto de una sola superred.

e12 Ejecutar las pruebas automatizadas

```
python -m pip install -r requirements-dev.txt
python -m pytest -q
python -m pip check
```

Resultado esperado para esta versión: 37 pruebas correctas y dependencias sin conflictos. Se comprueban los ejemplos, límites de hosts, capacidad insuficiente, ausencia de solapamientos, rechazo de entradas inválidas, comportamiento HTTP y agregación exacta. El cliente de pruebas puede emitir una advertencia de deprecación de Starlette sobre httpx; en la verificación registrada no produjo fallos.

La instalación de requirements-dev.txt también se verificó en un entorno virtual nuevo para comprobar la resolución de dependencias desde los archivos entregados. Las comprobaciones HTTP se realizaron con un proceso Uvicorn real. No se midieron carga, concurrencia máxima ni latencia y no se realizó una auditoría de seguridad.

e13 Instalar un servicio persistente en Ubuntu Server

Este procedimiento utiliza systemd y requiere permisos administrativos en el servidor de destino. Primero completar e1 y comprobar que /opt/netsegment no contiene otra instalación que deba conservarse.

Crear una cuenta de servicio sin inicio de sesión y clonar el código:

```
sudo useradd --system --user-group --no-create-home \
  --shell /usr/sbin/nologin netsegment
sudo git clone \
  https://github.com/DanielUCSM/X_NetSegment-API.git \
  /opt/netsegment
sudo python3 -m venv /opt/netsegment/.venv
sudo /opt/netsegment/.venv/bin/python -m pip install \
  -r /opt/netsegment/requirements.txt
```

Resultado esperado: usuario y grupo netsegment, código en /opt/netsegment y entorno con dependencias instaladas. Si la cuenta ya existe, comprobarla con id netsegment y omitir useradd.

Configurar el archivo local e instalar la unidad:

```
sudo install -o netsegment -g netsegment -m 600 \
  /opt/netsegment/.env.example /opt/netsegment/.env
sudo install -m 644 \
  /opt/netsegment/deploy/netsegment.service \
  /etc/systemd/system/netsegment.service
sudo systemctl daemon-reload
sudo systemctl enable --now netsegment
sudo systemctl status netsegment --no-pager
curl -fsS http://127.0.0.1:8000/health
```

Resultado esperado: servicio active (running), inicio habilitado al arrancar el servidor y disponibilidad HTTP 200 desde el mismo servidor. La unidad ejecuta el proceso con una cuenta restringida, reinicia

ante fallos y protege el sistema de archivos. El ejemplo conserva la escucha local; para clientes de otros equipos se requiere la configuración de acceso institucional.

Consultar registros y detener el servicio:

```
sudo journalctl -u netsegment -n 50 --no-pager
sudo systemctl stop netsegment
```

Resultado esperado: registros de Uvicorn y servicio detenido después de stop. Reiniciar con `sudo systemctl start netsegment` cuando corresponda. Una eventual publicación HTTPS y su autenticación son responsabilidad de la infraestructura y no están instaladas por estos comandos.

e14 Actualizar y recuperar una versión

Registrar el commit actualmente instalado antes de actualizar:

```
sudo git -C /opt/netsegment rev-parse HEAD
sudo git -C /opt/netsegment pull --ff-only
sudo /opt/netsegment/.venv/bin/python -m pip install \
-r /opt/netsegment/requirements.txt
sudo /opt/netsegment/.venv/bin/python -m pip check
sudo systemctl restart netsegment
curl -fsS http://127.0.0.1:8000/health
```

Resultado esperado: actualización sin sobrescribir cambios locales y servicio correcto. Si la validación falla, detener el servicio, restaurar el commit previamente registrado mediante `git checkout` y reinstalar sus dependencias antes de reiniciar. La recuperación no implica restaurar una base de datos porque el servicio no persiste información.

e15 Solucionar incidencias frecuentes

Síntoma	Causa probable	Acción y resultado esperado
No module named fastapi	Entorno inactivo o instalación incompleta	Activar <code>.venv</code> e instalar <code>requirements.txt</code> ; el módulo queda disponible
No module named app	Ejecución fuera de la raíz	Entrar a <code>netsegment</code> ; <code>python -m app</code> inicia el servidor
Address already in use	Puerto ocupado	Detener el otro proceso o editar <code>PORT</code> ; Uvicorn inicia en el puerto libre
Connection refused	Backend detenido o URL incorrecta	Iniciar el proceso y revisar <code>HOST/PORT</code> ; <code>/health</code> responde 200
HTTP 422	CIDR o campos inválidos	Comparar con <code>examples</code> y <code>/docs</code> ; una entrada válida responde 200
HTTP 400 en VLSM	Bloques requeridos exceden la red	Aumentar la red base o reducir demanda; asignación completa válida
HTTP 400 en CIDR	Solapamiento, discontinuidad o alineamiento	Corregir las redes; la unión debe ser una sola superred exacta
Servicio systemd failed	Ruta, dependencia, puerto o <code>.env</code> incorrectos	Revisar <code>journalctl</code> y la unidad; después corregir y reiniciar

f Repositorio GitHub

f1 Organización y contenido del repositorio

La estructura de la solución incluye el backend, README.md, requirements.txt, requirements-dev.txt, .env.example, .gitignore, ejemplos JSON, pruebas, script de comprobación HTTP, configuración systemd, contrato OpenAPI y este protocolo con su diagrama. No se incluyen migraciones porque la solución no usa base de datos. El informe técnico se incorpora en docs en Word y PDF.

El README presenta nombre, descripción, integrantes, tecnologías, requisitos y comandos básicos de instalación y ejecución. El código probado corresponde a la versión 1.0.0 definida en app/__init__.py. Durante la revisión debe registrarse git rev-parse HEAD y comprobar que /health declara la misma versión que el código del checkout.

f2 Nombre del repositorio y actualización del remote

La estructura obligatoria NumeroGrupo_TituloProyecto corresponde a 13_NetSegmentAPI. Para configurar ese nombre, el propietario debe abrir Settings, General, Repository name. Al cambiar el nombre, actualizar la copia local:

```
git remote set-url origin \  
https://github.com/DanielUCSM/13_NetSegmentAPI.git  
git remote -v
```

Resultado esperado: origin apunta a 13_NetSegmentAPI. Mantener los enlaces de README, este protocolo y el informe sincronizados con la URL del repositorio.

f3 Invitación al docente

El propietario o administrador debe abrir Settings, Collaborators, Add people y enviar la invitación a jangulo@ucsm.edu.pe. Verificar que el destinatario corresponda a Javier Fernando Angulo Osorio. Si GitHub exige un nombre de usuario y el correo no resuelve una cuenta, confirmar ese dato con el docente; no seleccionar otra persona por similitud.

Resultado esperado: invitación enviada o colaborador aceptado antes de la revisión. El comprobante se obtiene de la configuración del repositorio.

f4 Protección de información sensible

Solo se versiona .env.example. .gitignore excluye .env, entornos virtuales, cachés, claves y nombres comunes de archivos de credenciales. Antes de subir cambios:

```
git status --short  
git diff --cached
```

Resultado esperado: únicamente código y documentación revisados. La exclusión por nombre no reemplaza la revisión del contenido. No incorporar contraseñas, tokens, claves privadas, certificados privados ni credenciales de bases de datos. Si se expone una credencial, revocarla y tratar también el historial; borrar el archivo del último commit no revoca una clave.

f5 Criterios de aceptación de la entrega

Criterio	Comprobación
Instalación reproducible	Dependencias instaladas desde requirements.txt en un entorno nuevo

Criterio	Comprobación
Backend ejecutable	python -m app inicia y /health responde 200
Operaciones principales	Ejemplos FLSM, VLSM y CIDR con resultados previstos
Validación de errores	Entradas inválidas 422 y capacidad insuficiente 400
Correspondencia del código	Commit registrado y versión 1.0.0 coincidente
Documentación	README y protocolo disponibles con comandos y resultados
Nombre obligatorio	Estructura NumeroGrupo_TituloProyecto correspondiente a 13_NetSegmentAPI
Colaboración docente	Invitación a jangulo@ucsm.edu.pe enviada antes de la revisión
Datos académicos	Grupo 13, integrantes y curso Computación en Red III identificados

Las pruebas funcionales registradas validan el comportamiento de laboratorio. La revisión de la entrega debe comprobar la instalación reproducible, la correspondencia entre el código y la versión demostrada, la documentación y los criterios administrativos de la tabla.

Fuentes de verificación técnica

La información funcional y los resultados proceden del código, del contrato OpenAPI generado y del registro docs/VALIDACION.md. Los datos del equipo y el diseño inicial provienen del dossier tecnológico de septiembre de 2026 y del README existente. Las instrucciones de uso de IPv4Network y del servidor ASGI se contrastaron con documentación oficial:

Python Software Foundation. Documentación de ipaddress para Python 3.12. <https://docs.python.org/3.12/library/ipaddress.html>

FastAPI. Run a Server Manually. <https://fastapi.tiangolo.com/deployment/manually/>

Estas fuentes documentan interfaces y ejecución del software; no se presentan como evidencia de rendimiento ni como artículos científicos.